Module-I
Lecture-II
High Level Design Representation

1. **Introduction**

As discussed in the last lecture, almost all steps of VLSI design are automated. Any automated procedure requires that input data being provided is in some predefined format. Also, the models used to represent the inputs and transformations (changes of the input) should be efficient for execution of the procedure. For example, in case of High Level Synthesis (HLS) the input specifications are generally in some Hardware Definition Language (HDSs) like Verilog [1], VHDL [2], System C [3] etc. The HDL specifications are represented using several modeling paradigms like Control and Data Flow Diagram (CDFG) [4], DeJong's hybrid flow graph [5], SSIM flow graph [6], Finite state machine with data [7] etc., which are suitable for scheduling, allocation and binding procedures. Sometimes timing constrains (on execution of steps) are also given in the specifications, which are modeled by the above paradigms, however, with timing parameter included e.g., CDFG with timing, DF with timing and CF with timing.

In this lecture, we will discuss CDFG paradigm for modeling of high-level hardware descriptions (given in Verilog). CDFG is one of the most widely used modeling paradigm and the others mentioned above are not much different; for details of other paradigms the reader may look into the respective references.

2. **Control and Data Flow Diagram (CDFG)**

A CDFG is a directed graph $G = (V, E)$, where $V = \{v_1, \ldots, v_n\}$ is the set of nodes and $E = \{e_1, \ldots, e_m\} \subseteq V \times V$ the set of directed edges $e_i = (v_j, v_k)$.

In general, the nodes in a CDFG can be classified into one of the following types:

- Operational nodes: These are responsible for arithmetic, logical or relational operations (or computations); e.g., addition, equality checking etc.
- Control nodes: These nodes are responsible for control operations like conditions, loop constructs etc.; e.g., case statements, while loop etc.
- Storage nodes: These nodes represent assignment operations associated with variables and signals; e.g., reading an input value to register etc.

The edges in a CDFG represent:

- Transfer of values (in variables that are changed due to processing in operational and storage nodes). A node needs data generated by its predecessor nodes and generates new data needed by its successors. Nodes operate on the data of the incoming edges. The resulting data is put on the outgoing edges.
- Control flow from one node to another. An edge can also represent a condition, e.g., while implementing loop constructs, if/case statements etc.

Now we illustrate different types of nodes and edges using a simple example. Let us take the simple Verilog HDL code given in Figure 1.

```
module CDFG_example (A,B,C,D);
input [3:0] B,C,D;
 reg [7:0] A;
output [7:0] A;
        initial begin A = B * C + D; end
        while ( A > 0 ) begin
        A : = A − 1;
          end
endmodule
```

Figure 1. Verilog code

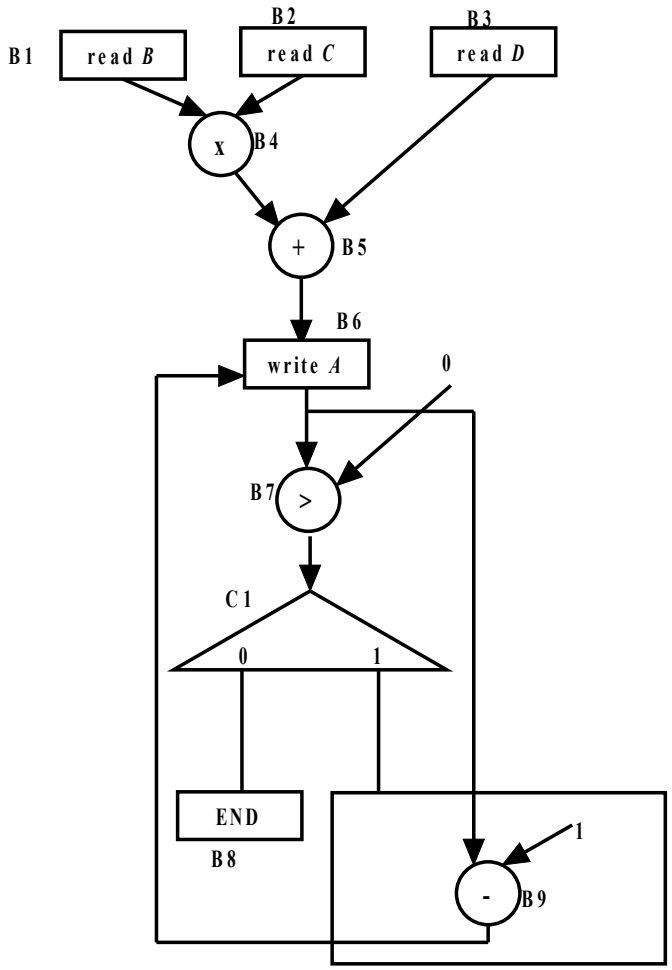


Figure 2. CDFG for the Verilog HDL in Figure 1

The CDFG for Verilog HDL (Figure 1) is shown in Figure 2. In Figure 1 it may be noted that there are three input variables (B,C and D) which must be read from input lines to registers. So corresponding to reading of each variable in registers we have a storage nodes; B1,B2 and B3 are storage nodes (Figure 2). In the Verilog code there is a one time computation "initial begin A: = B * C + D; end". For this computation we see that there are 2 sub-computations, namely "*" and "+". So we have two operational nodes, B4 and B5 for "*" and "+", respectively. The edge $\langle B1, B4 \rangle$ corresponds to transfer of value of B, which gets changed (i.e., new value read) due to processing (reading) in B1. The edge $\langle B4, B5 \rangle$ corresponds to transfer of values (B and C to "B*C"), which get changed due to processing ("*") in B4. After the

computation “initial begin A: = B * C + D; end”, the value is stored in A; this is captured by storage node B6. B7 is the operational node that checks 0 with A; output is 0 if $A<0$ and 1, otherwise. The output of B7 (carried by edge $\langle B7, C1\rangle$) controls the control node C1. $\langle B7, C1\rangle$ is the control flow edge, which corresponds to the condition (A>0) required for execution/exit of the while loop. Node C1 is a control node responsible for deciding data flow direction after the condition “A>0” is checked at B7. If value transferred by $\langle B7, C1\rangle$ is 0 then the loops exits at B8, else computation “A=A-1” is done at B9 and the loop continues.

Now we discuss in brief, CDFGs for some other frequently used constructs of HDLs. Figure 3 illustrates CDFG for “case” statement in Verilog. This is similar to if-then statement; however, we have an edge for all the cases of the case statement in the control node.

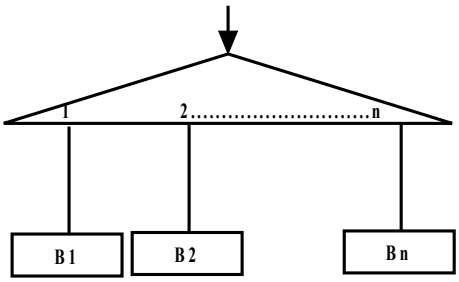


Figure 3. CDFG for “case” statement in Verilog

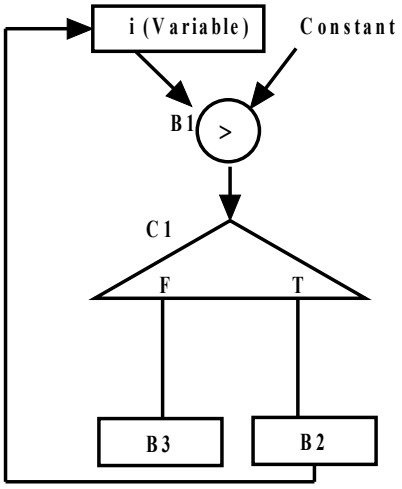


Figure 4. CDFG for “for loop” in Verilog

The CDFG for “for loop” in Verilog is shown in Figure 4. The CDFG has a storage node (B1) which holds the variable on which the loop executes. It also has an operational node that compares variable with constant (or any other variable) to decide execution/exit of the loop. The output of this operational node feeds a control node (C1) which decides data flow direction after the condition is checked. Finally, the graph has two other operational nodes (B3 and B4) which correspond to data flows from the control node.

From the discussion, it is clear that CDFGs are quite coherent with HDLs and can be easily built from hardware specifications. In this lecture, we will not discuss on other HDL statements (which can be built in a similar way as discussed above), for details, the reader is refereed to [4].

3. **CDFG with timing**

Sometimes minimum required delay needs to be mentioned in the specifications. Delay information is required in logic synthesis. Delay of a circuit depends on gates and architecture. For example, speed of ripple-carry adder is lower compared to carry-look-ahead adder; however, area of carry-look-ahead adder is higher than ripple-carry adder. Also, faster gates consume more area and power compared to slower gates. So, if tolerable delay of some operation in a circuit is low (which depends on application), realizing it with faster circuitry unnecessarily leads to high area and power overheads. In CDFG, delay information is modeled using a node between the required points, where the delay is specified. The points can be between single operational nodes (shown in Figure 5), between multiple operational nodes (shown in Figure 6), loop (shown in Figure 7) etc.

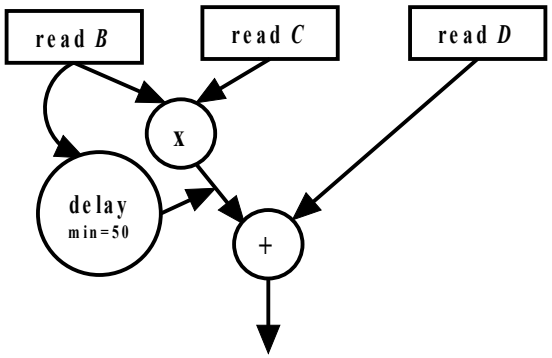


Figure 5. CDFG with timing requirements between a single operational node

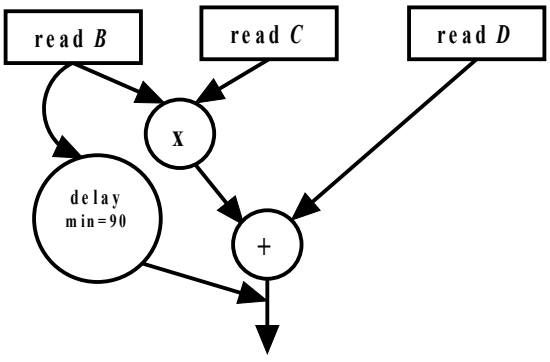


Figure 6. CDFG with timing requirements between multiple operational nodes

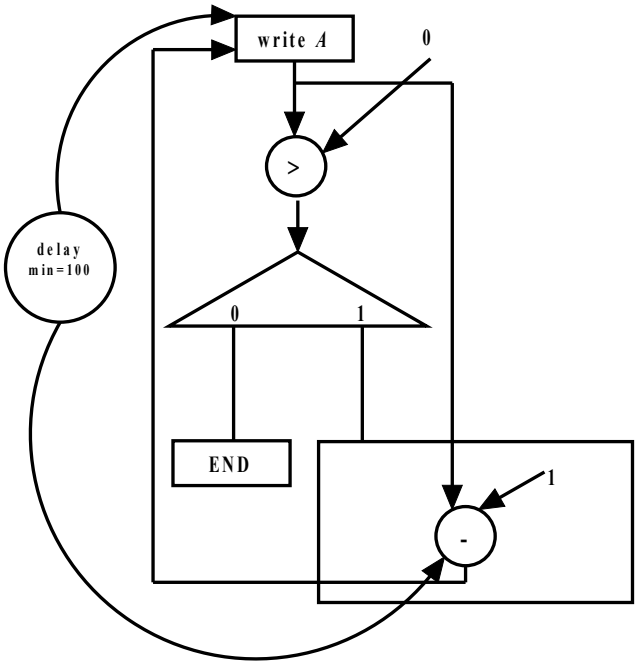


Figure 7. CDFG with timing requirements in loops

4. **CDFG: Control flow based representation and Data flow based representation**

CDFG can be control flow based or data flow based. In this section, we will illustrate these two types of representations using an example of a Verilog code; the code is shown in Figure 8. The Verilog code involves some computations over inputs B,C and D depending on value of "*cond*".

```
module example1(A,B,C,D,cond);
input [3:0] B,C,D;
input [1:0] cond;
 reg [3:0] A;
output [3:0] A;
        case (cond)
           00: A = B + C + D;
           01: A = B - C + D;
           default: A = B + C - D;
        endcase
endmodule
```

Figure 8. Verilog code having case statement

The control flow based CDFG for the code is shown in Figure 9. The CDFGs discussed in the last section were control flow based ones. From Figure 9 it may be noted that for each value of "*cond*" there is a sub-sequence of operations represented by storage and operational nodes. Here, the operations are classified based on three values of "*cond*": 00, 01, default (10,11) and the corresponding sub-sequence of operations involving storage and operational nodes are enclosed by three squares. Therefore, the control flow based CDFG has almost a one to one mapping with the lines of Verilog code. So, control flow based CDFG gives an idea that, depending on value of condition of a control node ("*cond*" in this

example) only one sub-set of operators and storages are executed. It may be noted that this is software concept because in a program only those parts of a code are executed which satisfy conditional statements like “if-then-else”, “case” etc. However, in hardware there is no concept of execution of a “sub-set” of operators/storages (i.e., statements of an HDL code), based on value of condition of a control node. In the example of Verilog code in Figure 8, variable “A” can be assigned three different values through three different computations depending on value of “*cond*”. So, three hardware circuitry are to be kept in the chip and depending on the value of “*cond*”, the output of appropriate circuitry would write “A”. In other words, unlike a software code, where parts of a code can be invoked based on values of a condition, in hardware, all the different types of circuitry are to be implemented in the chip (and would be executed) and the output being used depends on the value of a condition.

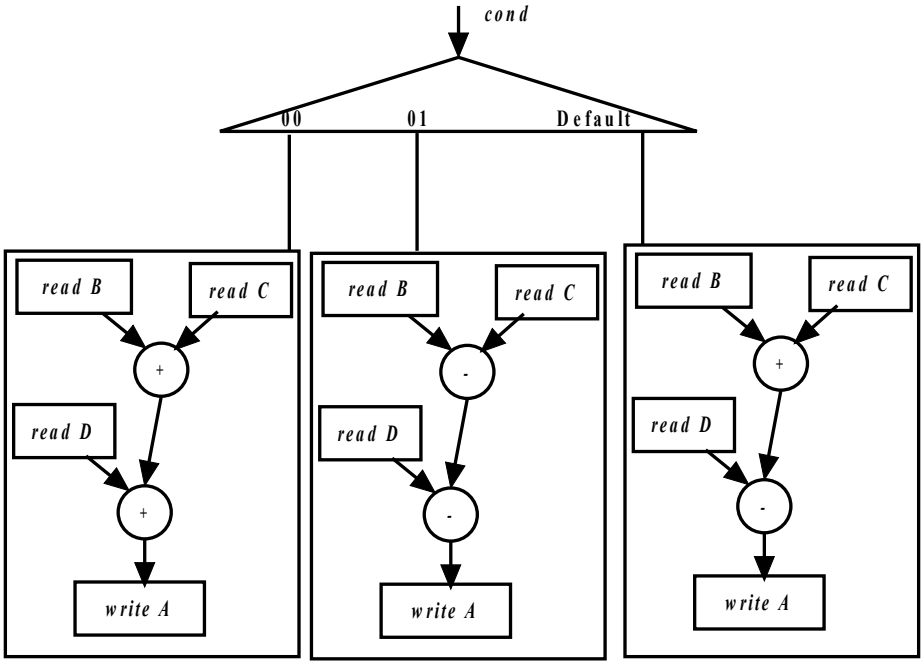


Figure 9. Control flow based CDFG for Verilog code of Figure 8

CDFG that uses the above-mentioned concept of hardware is called data flow based representation. Figure 9 illustrates data flow based CDFG for Verilog code of Figure 8. It may be noted that in this case, the control node is after the operational and storage nodes. There are three circuits for computing different

values of “A" , depending on “*cond*”. The three circuits evaluate three different outputs irrespective of the value of “*cond*”; “A” is written by the output of the appropriate circuit depending on value of “*cond*”. It may also be noted that even if three different circuitry are required to compute different possible values of “A”, many operational and storage nodes are common among these circuits and redundancy can be eliminated. For example, reading of B,C,D are required by all the three circuits and they may be implemented by storage nodes common to all the three circuits. Also, operational node for “B+C” is common to circuit for “A = B + C + D” and circuit for “A = B + C - D”, which can be merged.

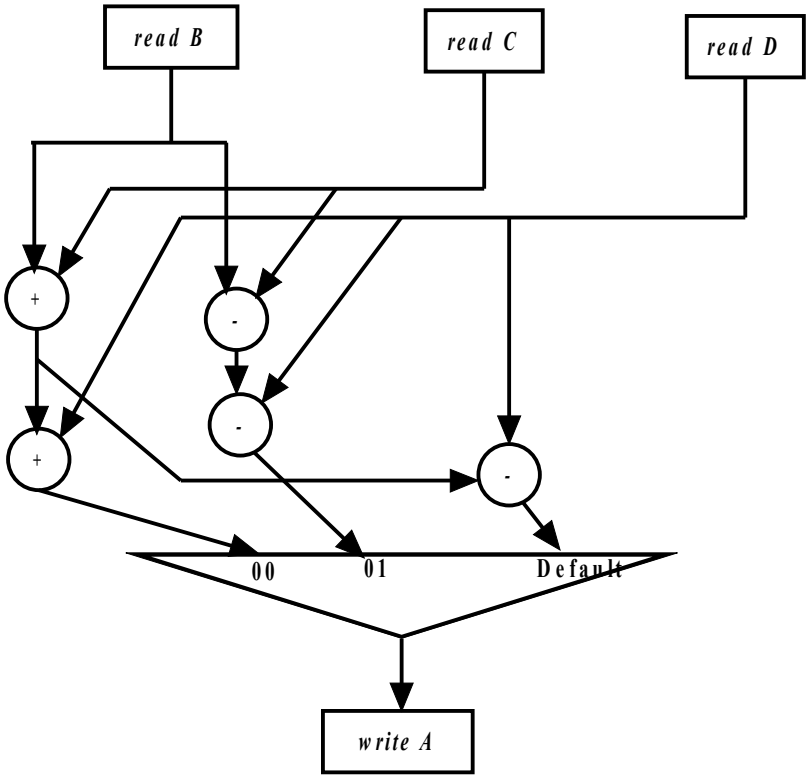


Figure 9. Data flow based CDFG for Verilog code of Figure 8

5. **Conclusions**

In this lecture is we have stated the concept that for automation, input data is to be represented using appropriate models. In case of HLS, the first step of VLSI design flow, several modeling paradigms exist like CDFG, FSM, FSMD, DeJong's hybrid flow graph, SSIM flow graph etc. In this lecture, we have discussed CDFG framework in details and illustrated with several examples. Following this modeling step, the CDFGs need to transformed so they become more amenable for HLS tasks like scheduling, allocation and binding. The next lecture focuses on transformations of CDFG for HLS.

Question and Answers

1. What are the advantages and disadvantages of Control flow based CDFG over Data flow based CDFG?

Answer:

In data flow based CDFG, we can represent parallel evaluation by operational and storage nodes for all branches of a control node. This is in real sense more near to hardware realization of the circuit, compared to that of the control flow based CDFG, where only that set of operational and storage nodes are executed for which (branch) the value of control node is satisfied. Therefore, optimizations and other steps of HLS are more suitable to be performed on data flow based CDFG.

On the hand, data flow based CDFG is more tedious to draw from the specifications compared to the control flow based counterpart. In control flow based CDFG there is almost one-to-one mapping between the nodes of the CDFG and the lines of DHL code. If there are nested loops and conditions in the input specifications then it should be flattened before extracting the data flow based CDFG. Further, in such cases the CDFG may have a large number of nodes.

References

[1]. Samir Palnitkar, “Verilog HDL: A guide to digital design and synthesis”, 2nd Edition, Prentice Hall Press, 2003

[2]. Peter J. Ashenden, “The Designer's Guide to VHDL”, 2nd Edition, Morgan Kaufmann Publishers Inc., 2001

[3]. Wolfgang Muller, Wolfgang Rosenstiel and Jurgen Ruf, “SystemC: methodologies and applications”, 1st Edition, Kluwer Academic Publishers, 2003.

[4]. S.Amellal and B.Kaminska. ”A CDFG model for synthesis from VHDL”. Technical report, Ecole Polytechnique de Montreal, August 1997.

[5]. Gjalt G. de Jong, “Data flow graphs: system specification with the most unrestricted semantics”, Proceedings of the conference on European design automation, pages 401--405, 1991.

[6]. R. Camposano and R. M. Tabet, “Design representation for the synthesis of behavioral VHDL models”, Symposium on Computer Hardware Description Languages and Their Applications, page 49 - 58, 1989.

[7]. D. Gajski and L. Ramachandran, “Introduction to high-level synthesis”, IEEE Design and Test of Computers, Vol. 11, No. 4, page 44-54, 1994.